

Starting 7:05

Today's Agenda:-

1. Power function
2. All indices
3. Tower of Hanoi

Q: 1. Implement Power function ?
Given 2 integer a & n .
Return a^N .

ex:-

$$a = 2; n = 3; \quad 2^3 = \underline{8}.$$

Idea 1:-

$$a^N = \underbrace{a * a * a \dots a}_{N \text{ times}} = a^{N-1} * a$$

Ass'n

Given two integers a & N . Cal
 a to the power N .

int pow(a, N);

Main Logic

$$\text{pow}(a, N) = \text{pow}(a, N-1) * a;$$

Base Con'n

if ($n == 0$) { return 1; }

Code

```
int pow(int A, int N) {  
    if (N == 0) { return 1; }  
    return pow(A, N-1) * A;  
}
```

Dry Run ($a = 5, n = 3$) ; $5^3 = \underline{125}$.

printf(pow(5, 3))

o/p:- 125.

$1 \rightarrow 125$
 $5 \rightarrow 5$
 $3 \rightarrow 3$

```

int pow (int a, int n) {
    if (n == 0) return 1;
    return pow(a, n-1) * a;
}

```

25

$5 \rightarrow 5$
 $2 \rightarrow 2$

```

int pow (int a, int n) {
    if (n == 0) return 1;
    return pow(a, n-1) * a;
}

```

$5 \rightarrow 5$
 $1 \rightarrow 1$

```

int pow (int a, int n) {
    if (n == 0) return 1;
    return pow(a, n-1) * a;
}

```

$5 \rightarrow 5$
 $0 \rightarrow 0$

```

int pow (int a, int n) {
    if (n == 0) return 1;
    return pow(a, n-1) * a;
}

```

T.C. $\therefore O(N)$

S.C. $O(N)$

Idea 2:-

$$a^N = \underbrace{a * a * a \dots a}_{a^{N/2}} * \underbrace{a * a * a}_{a^{N/2}}$$

① If N is even

$$\begin{aligned} 2^8 &= 2^4 * 2^4 \\ 3^{10} &= 3^5 * 3^5 \end{aligned}$$

② If N is odd

$$\begin{aligned} 2^9 &= 2^{9/2} * 2^{9/2} * 2 = 2^{4+4+1} \\ 3^{11} &= 3^{11/2} * 3^{11/2} * 3 = 3^{5+5+1} \end{aligned}$$

Generalising:-

If (n is even)

$$a^N = a^{N/2} * a^{N/2}$$

$$\text{pow}(a, N) = \text{pow}(a, N/2) * \text{pow}(a, N/2)$$

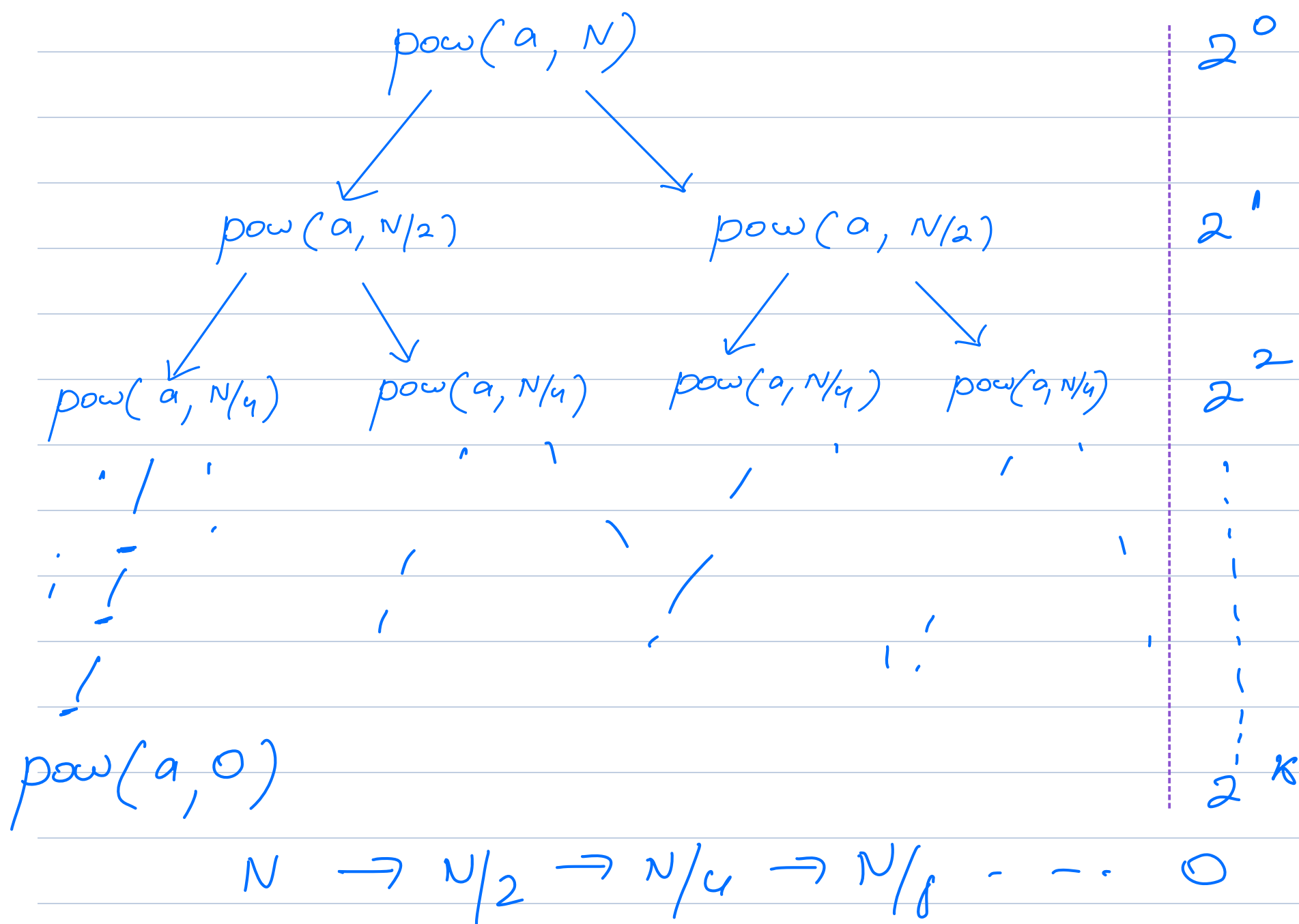
else

$$a^N = a^{N/2} * a^{N/2} * a;$$

$$\text{pow}(a, N) = \text{pow}(a, N/2) * \text{pow}(a, N/2) * a$$

Code

```
int pow (int A, int N) {  
    if (N == 0) { return 1; }  
    if (N % 2 == 0) {  
        return pow(a, N/2) * pow(a, N/2);  
    } else {  
        return pow(a, N/2) * pow(a, N/2) * a;  
    }  
}
```



S.C. :- $O(\log(N))$

T.C. : $2^0 + 2^1 + 2^2 + \dots + 2^k = O(N)$

$$= a \frac{(x^N - 1)}{x - 1}$$

$$= 2^0 \frac{(2^{K+1} - 1)}{2 - 1}$$

$$= 2 \cdot 2^K - 1$$

$$= 2^{\log_2 N}$$

$$a^{\log_a x}$$

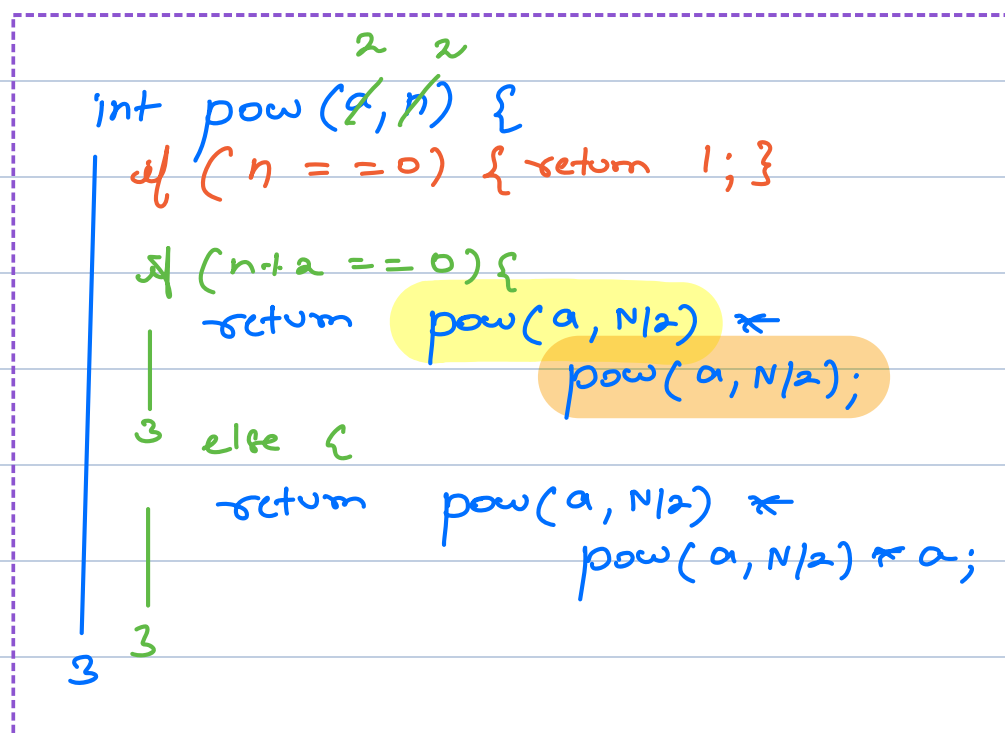
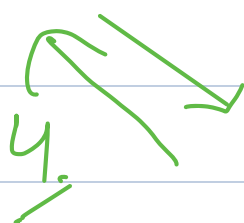
$$= N$$

Dry Run:-

(a = 2 ; n = 2)

$2^2 = 4$

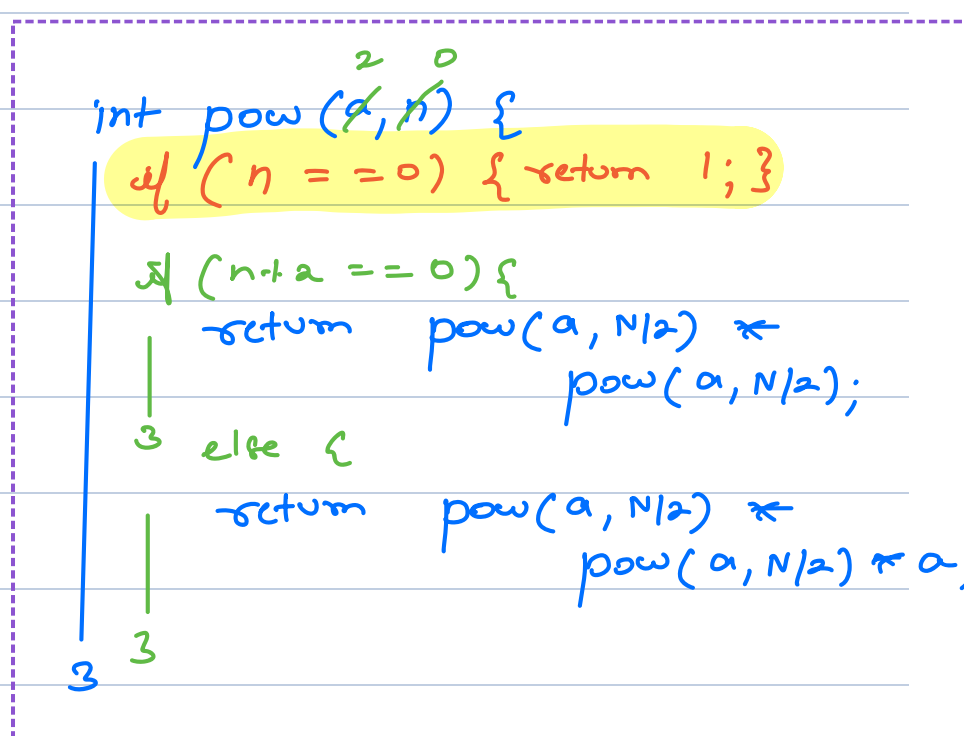
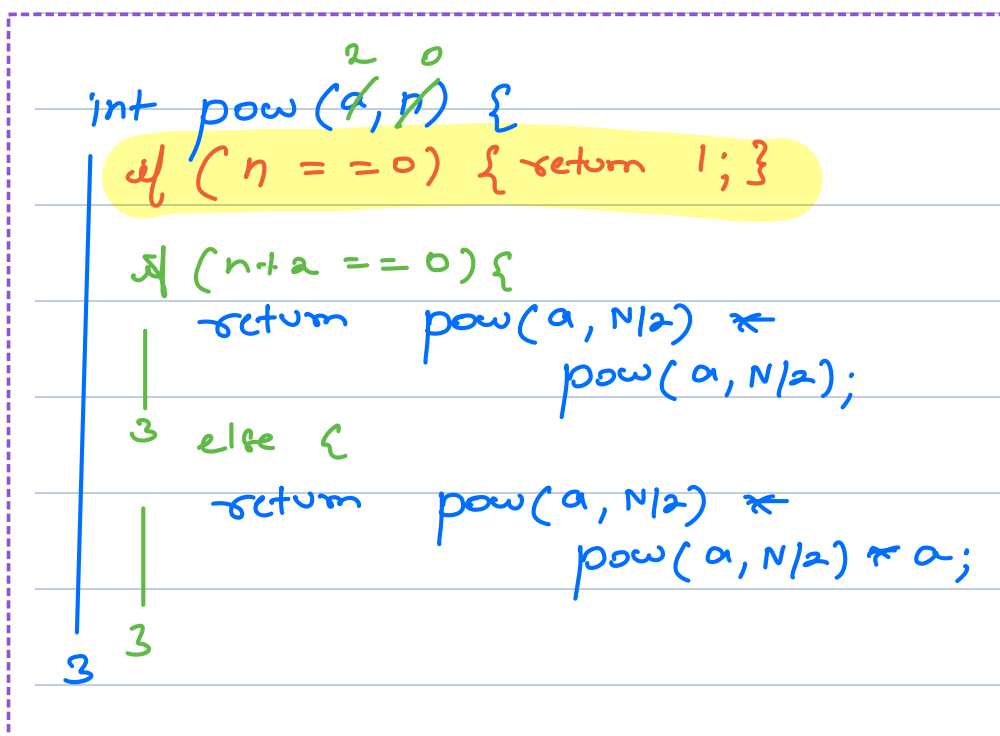
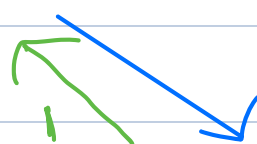
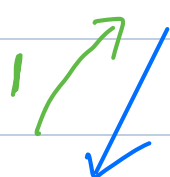
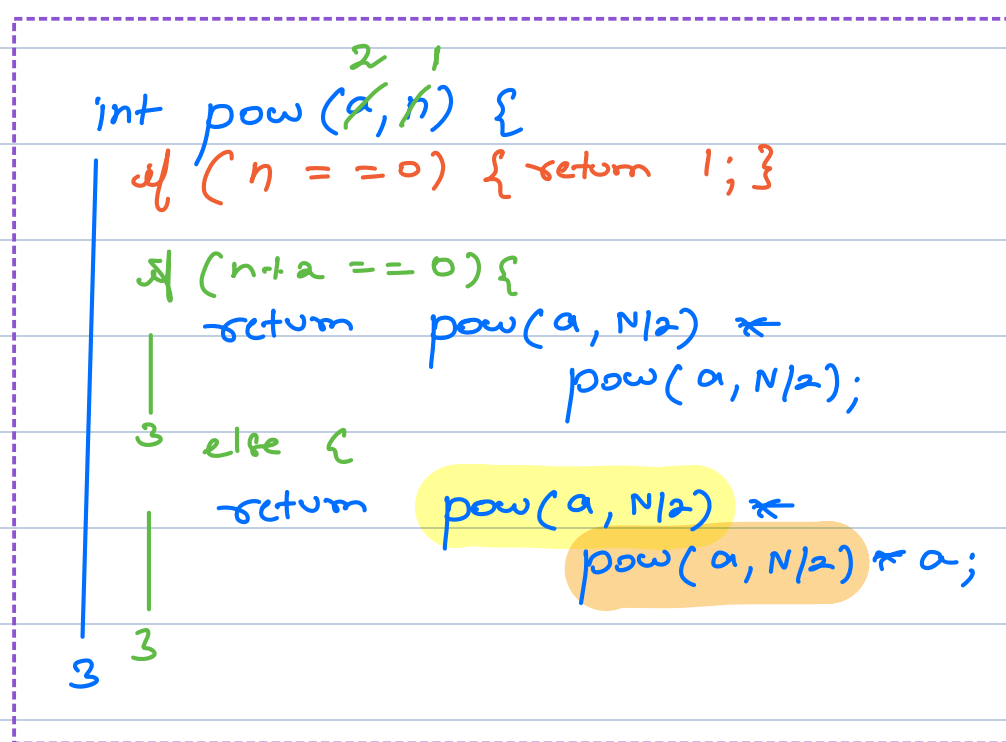
print(pow(2,2))



O/p:- 4



Some structure as LHS



Internal impl. of pow function

Further Optimisation

(Fast Power Algo)

Store the val. of $\text{pow}(a, N/2)$ to avoid calculating it again.

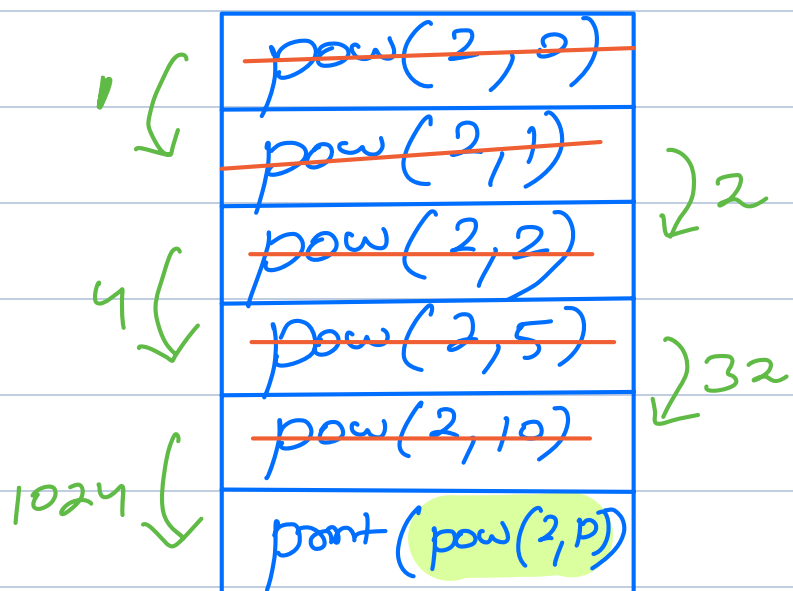
$$S.C \rightarrow O(\log N)$$

$$T.C \rightarrow O(\log N)$$

Code

```
int pow (int a, int N) {  
    if (n == 0) { return 1; }  
    p = pow (a, N/2);  
    if (n % 2 == 0) {  
        return p * p;  
    }  
    else {  
        return p * p * a;  
    }  
}
```

$$a = 2; N = 10;$$



$$32 * 32 = 1024$$

$$\underline{\underline{O/p :- 1024.}}$$

call stack

Q: 2

All indices of an array

Given an array A of size N & a target integer B . Return all indices where B is present in the array.

ex:- $\{4, 5, 3, 1, 5, 4, 5\}$ $B=5$
0 1 2 3 4 5 6
0 1 1 1 2 2 3

1	4	6
---	---	---

o/p $\rightarrow \{1, 4, 6\}$.

Quiz

$A = \{1, 2, 3, 1, 1\}$; $B=1$

o/p $\rightarrow \underline{\{0, 3, 4\}}$

Approach 1:- Use a dynamic array to keep track of all the indices having the element B .

Code

```
int[] solve (int[] ar, int B) {
```

```
    vector<int> ans;
```

OR

```
    ArrayList<Integer> ans;
```

```
    solution(ar, B, N, 0, ans);
```

```
    return ans;
```

```
}
```

```
void solution (int[] ar, int B,
```

```
               int N, int idx,
```

```
               ArrayList<Int> ans){
```

```
    if (i == N) { return; }
```

```
    if (ar[i] == B) {
```

```
        |         ans.add(i);
```

```
        |
```

```
        |         solution(ar, B, N, i+1, ans);
```

```
    }
```

Stricter Constraints:-

1. I can only use a static array.

2. I can only call recursion once; (Access every index only once).

Idea:-

1. We need to keep track of the size of result to create a static array.
  USE a counter

HINT:- Parent calls remain in stack & wait for child process to complete execution.

HINT:- We can return from a function :-
 a) Using return keyword.
 b) At the end of execution.

Approach:-

indices

1. Keep track of the count of indices equal to B while making recursive calls.
2. At the base condition since count is known, create a static array of size count.
3. While coming back, populate the indices in that static array.

Code

int[]

allIndex (int[] arr, int B,
int N, int i,
int cnt) {

① if (i == N) {

② | return new int[cnt];

③ if (arr[i] == B) {

④ | int A[] = allIndex (arr, B, N, i+1,
⑤ | A[cnt] = i; cnt+1);

⑥ | return A;

⑦ | else {

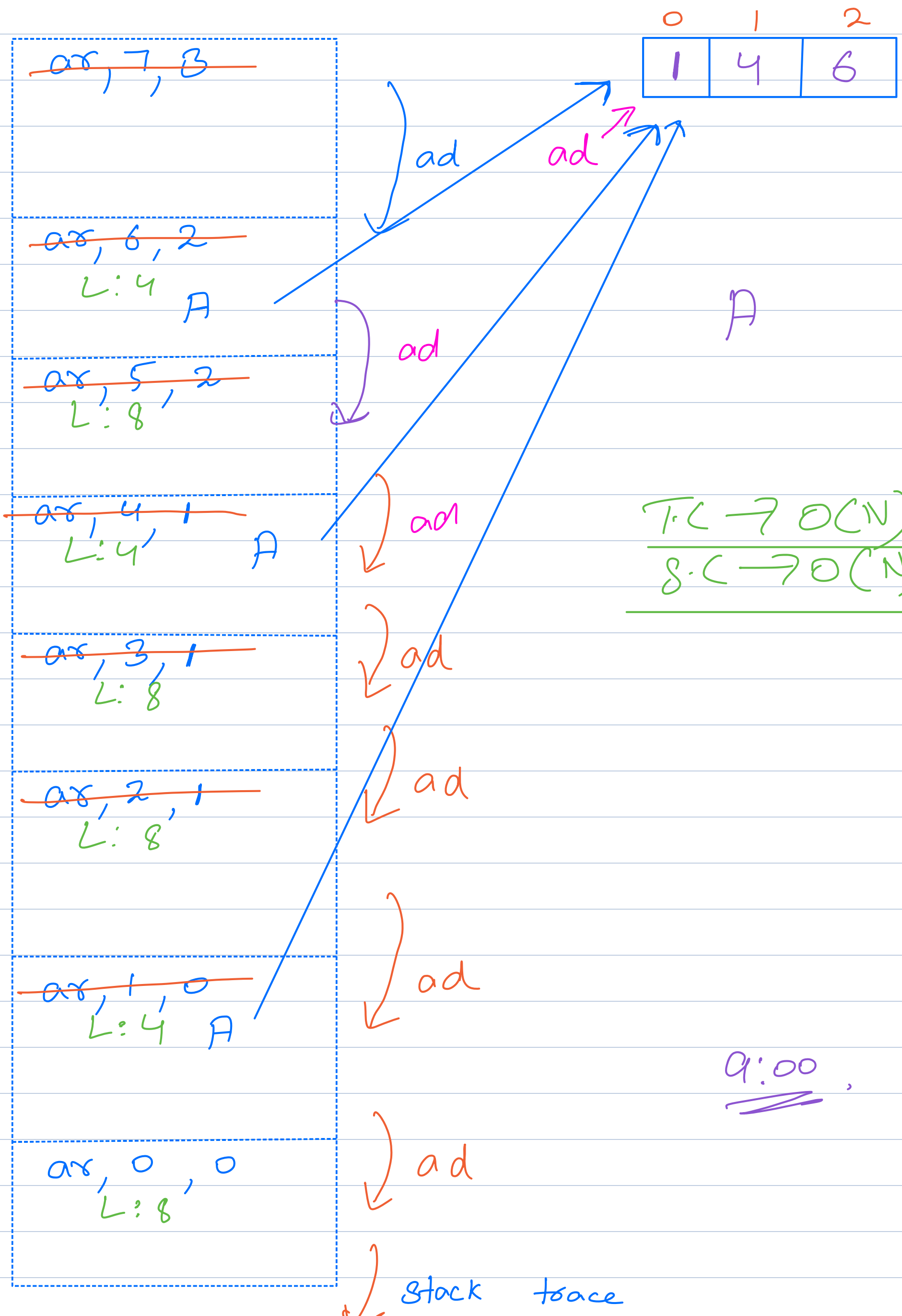
⑧ | return allIndex (arr, B, N, i+1,
| cnt);

3

Day Run

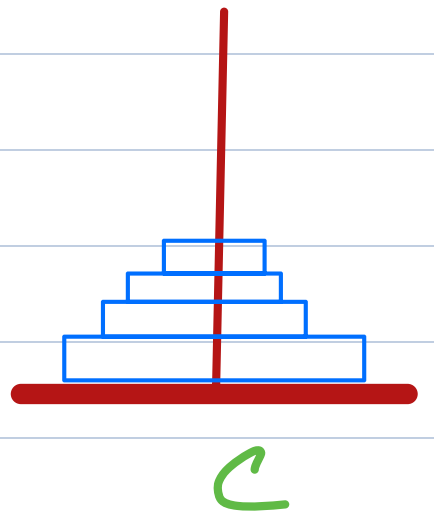
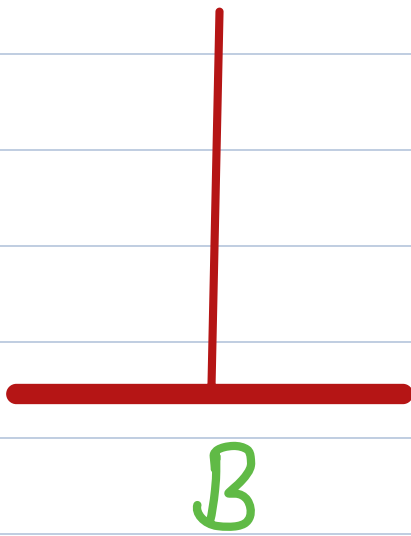
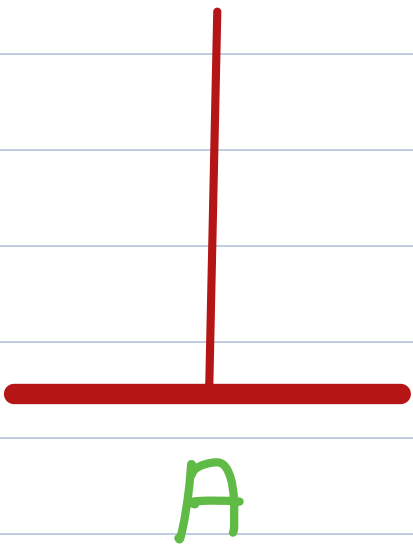
$N = 7$

$A = \{ \overset{0}{4}, \overset{1}{5}, \overset{2}{3}, \overset{3}{1}, \overset{4}{5}, \overset{5}{4}, \overset{6}{5} \} \quad B = 5$

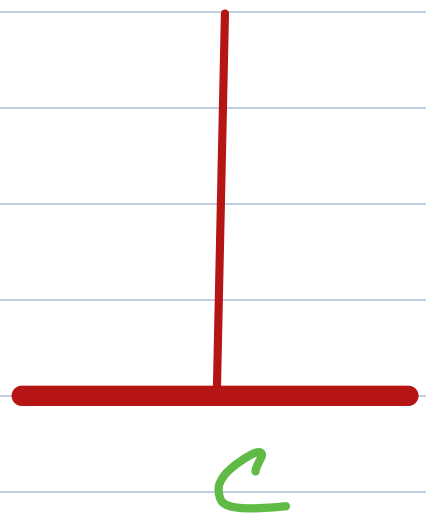
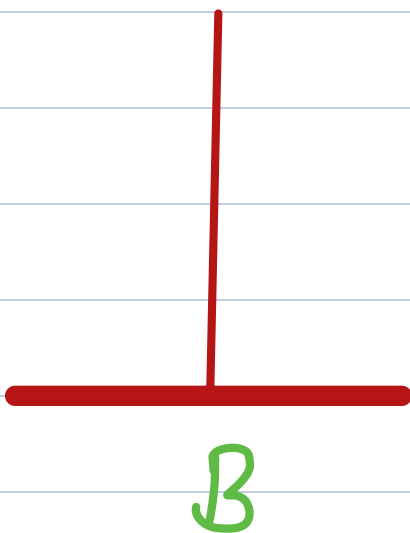
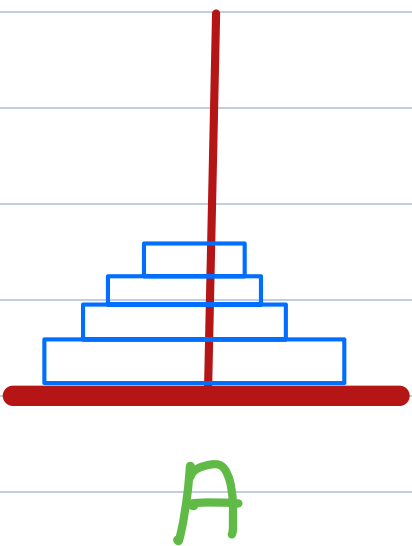


3. Tower of Hanoi — 3 towers A, B & C

There are N disks placed on tower A of different sizes.



Goal:- Move all the disks from tower B to C using tower A.



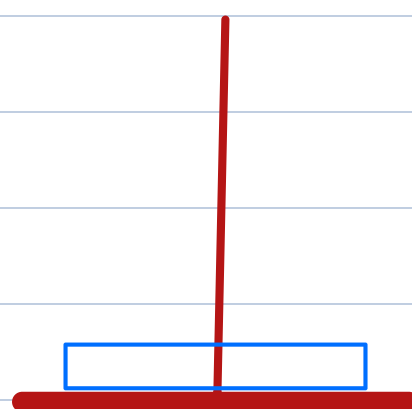
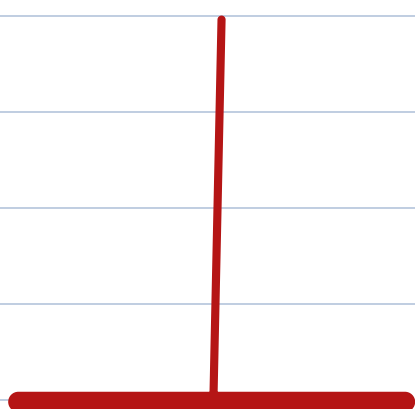
Rules:-

- ① A larger disk cannot be placed on a smaller disk.
- ② Only 1 disk can be moved at a time.

Point out the movement of disks.
(Minimum Steps)

Ex:- $N = 1$

1 A C

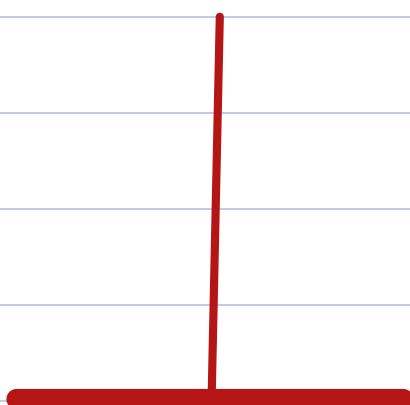
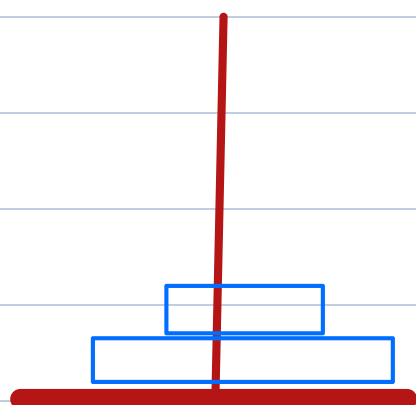


A

B

C

Ex:- $N = 2$



A

B

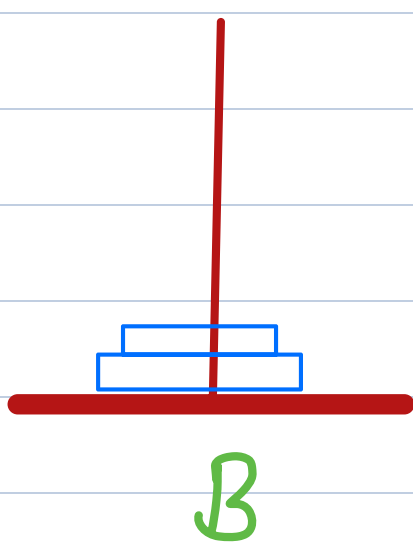
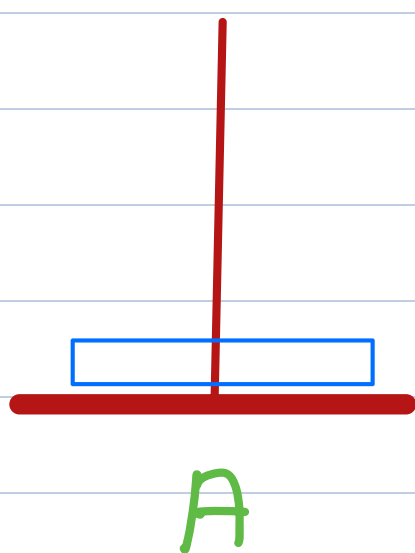
C

1 A B } move 1 disk from A to B + using C as a helper

2 A C } move 2nd disk directly from A to C.

1 B C } move 1 disk from B to C using A as a helper

Ex:- $N=3$



1	A	C	} Place first 2 disks from A to B using tower C.
2	A	B	
1	C	B	

3 A C → N A C

1	B	A	} Place first 2 disks from B to C using tower A.
2	B	C	
1	A	C	

Obs:-

$N-1^{\text{th}}$ → $N-1$
 $N-2^{\text{th}}$ → $N-2$

1. The largest disk should always be directly placed at TC directly from TA.

2. At the point when largest (N^{th}) disk is being moved from A to C, remaining $N-1$ disks should lie on top of tower B.

void toh (int N, ^{sr}A, ^hB, ^dC) {

if (N == 0) return;

Swaps helper to dest

toh (N-1, A, C, B);

print (Nth disk moved from
sr to d);

toh (N-1, B, A, C);

Swaps source & helper.

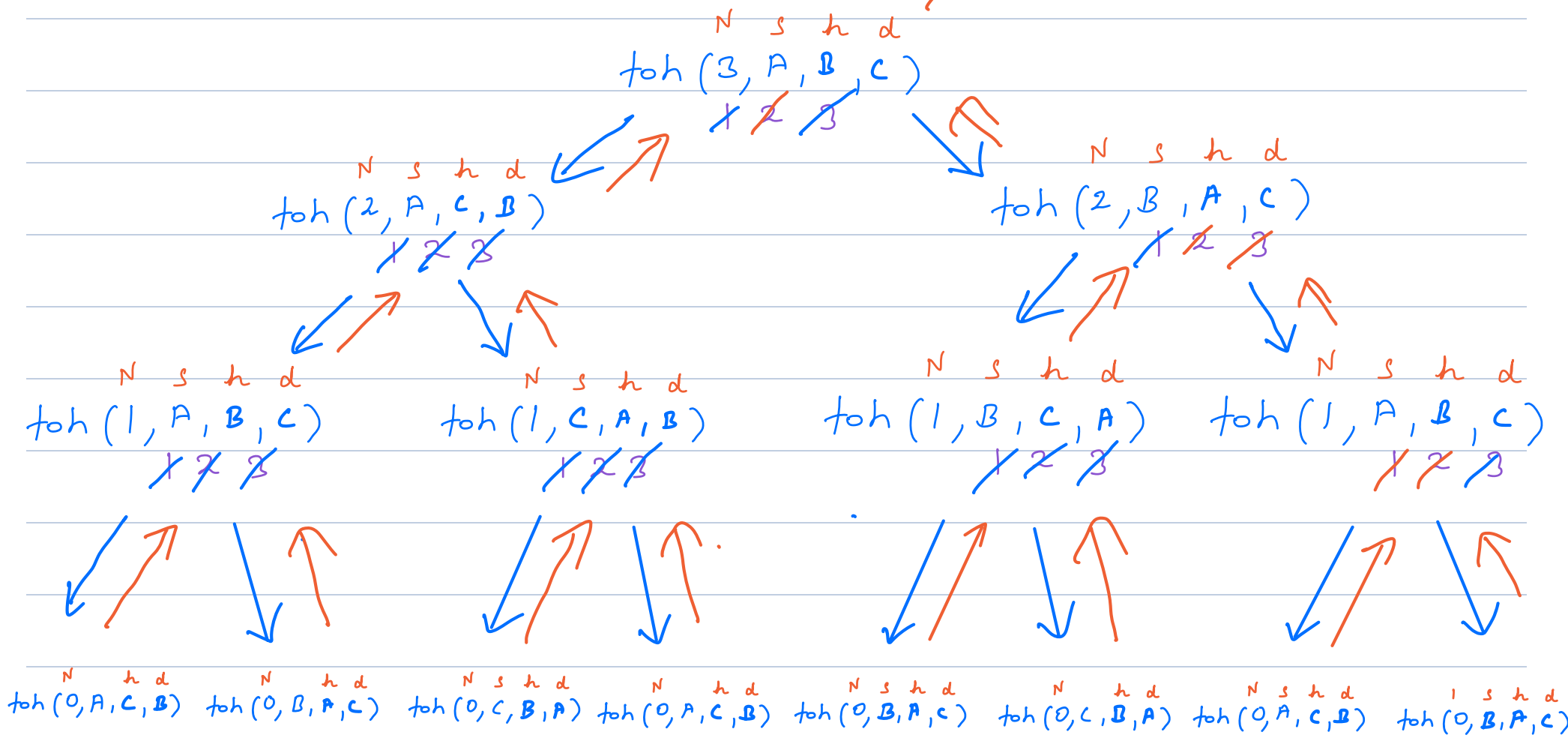
①

②

③

3

Dry Run $\rightarrow$ $N=3$



1 A C
2 A B
1 C B
3 A C
1 B A
2 B C
1 A C

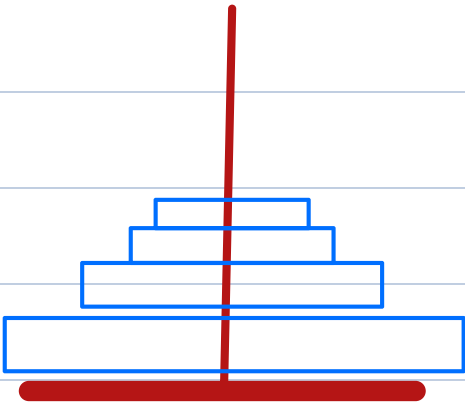
1 A C
2 A B
1 C B
3 A C
1 B A
2 B C
1 A C

H.W

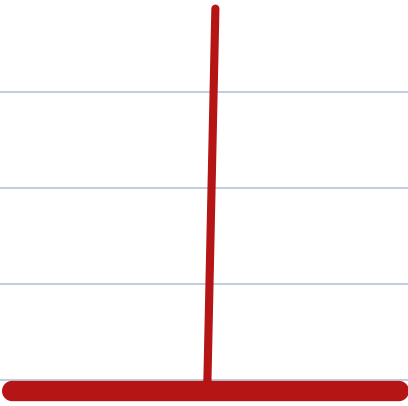
Dry Run $\rightarrow$ Yes

$$S.C = O(N)$$

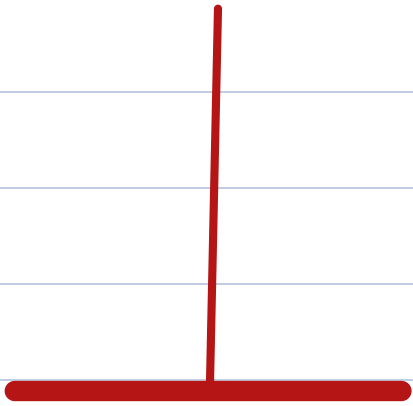
$$T.C = O(2^N)$$



A



B



C